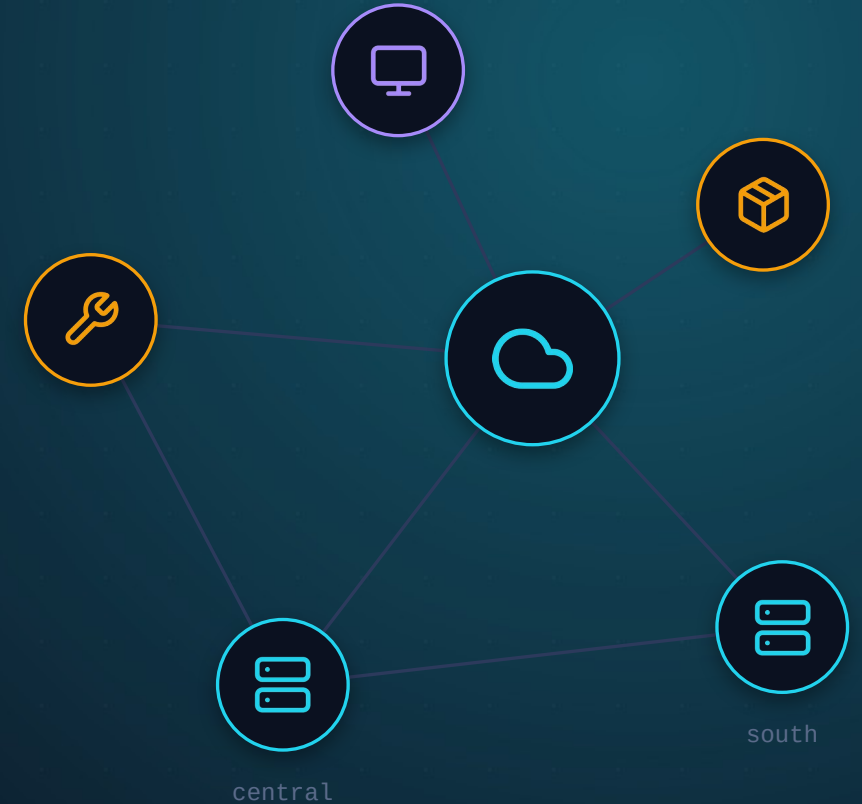


SERVERLESS ON CLOUDLET

From Knative to the Portal

What we built, how it fits together, and why we built it this way.

Team review · September 2026



The journey, in four chapters

Fourteen questions, one platform. We start at the idea and end at the console.

01



Foundations

- What is serverless
- What is Knative
- The OpenShift Serverless Operator
- Why we chose it

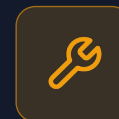
02



The API

- Purpose and structure
- Talking to the cluster with mTLS
- Active/active across two regions
- The tenant controller

03



Functions & builds

- Why functions are different
- Buildpacks and kpack
- A build, step by step
- The build controller

04



Portal & future

- How the portal wires it together
- The portal beyond serverless
- cloudlet-apis: the next API

What is serverless?

You bring code. The platform brings the servers, the scaling, and a bill only for what actually runs.



Scale to zero, and back

An idle workload costs nothing. The first request wakes it; a burst fans it out to as many replicas as the traffic needs.



No servers to own

No VMs to patch, no capacity to plan, no cluster to learn. You deploy a unit of work, not a machine.

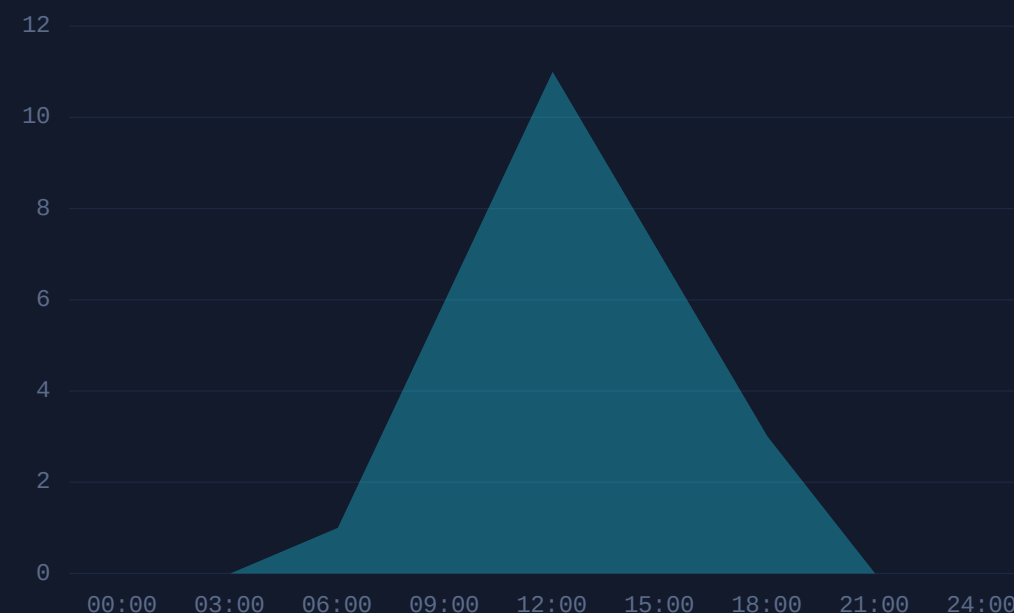


Driven by requests

Every instance exists because traffic asked for it, and disappears when the traffic stops. The unit of billing is the request, not the hour.

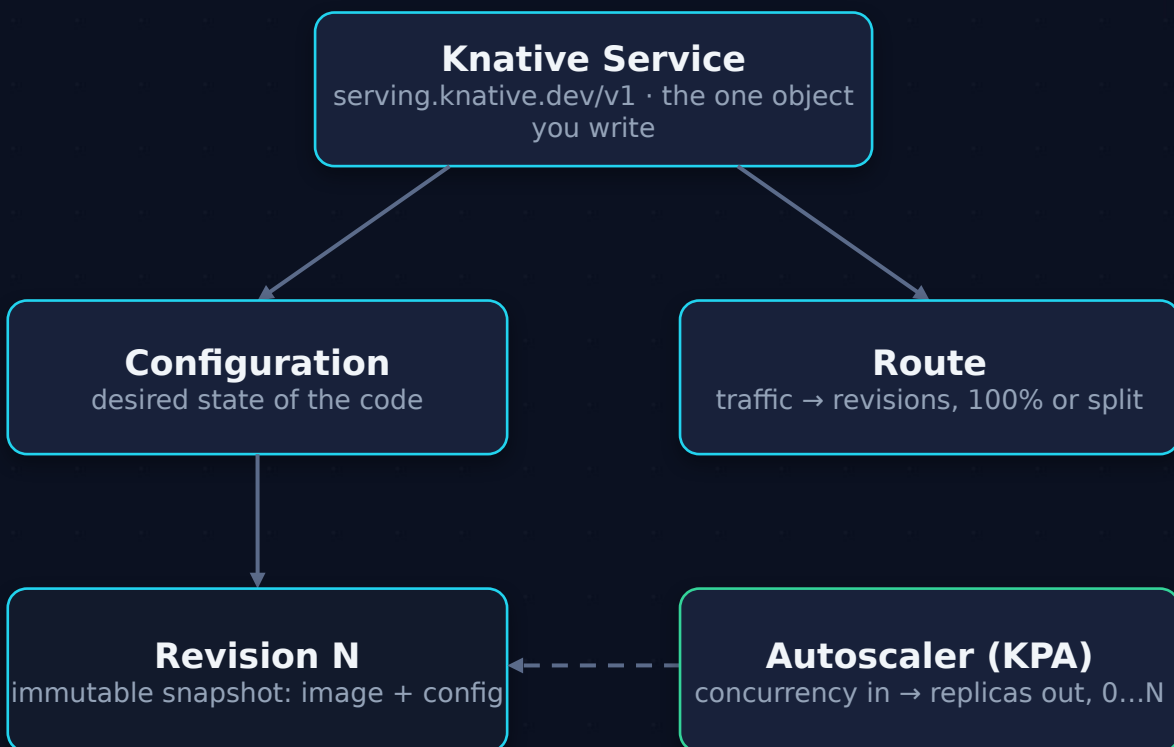
Replicas follow traffic

A function's day: nothing, a burst, nothing again.



What is Knative?

The Kubernetes-native building blocks for serverless. We use Serving: one object in, a routed, autoscaled, versioned workload out.



The Activator holds requests while a revision scales up from zero.



Serving

Request-driven workloads with scale-to-zero, revision history and traffic routing. The half we run in production.



Eventing

Brokers, triggers and CloudEvents for event-driven flows. Knative ships it; our platform does not expose it yet.



DomainMapping

Your own hostname on a Service. It becomes the way we give one workload the same address in both regions.

What is the OpenShift Serverless Operator?

Red Hat's supported distribution of Knative, installed, configured and upgraded by the Operator Lifecycle Manager.



One CR describes Knative

A KnativeServing custom resource in the knative-serving namespace. The operator installs every component from it and upgrades them with the catalog.



Kourier ingress and real Routes

Ingress runs in knative-serving-ingress. Every Knative Service and DomainMapping gets an OpenShift Route created for it, edge-terminated with the cluster wildcard.



Supported and mirrorable

OLM catalog images mirror with oc-mirror, which is how a supported Knative reaches an airgapped datacenter at all.

Knative Serving

Service · Revision · Route · autoscaler · DomainMapping

KnativeServing CR

one document: ingress class, domains, autoscaling defaults

OpenShift Serverless Operator

reconciles the CR, owns upgrades

Operator Lifecycle Manager

Subscription → CSV → catalog

OpenShift

the cluster, in each region

Why the Operator, not upstream Knative

Every alternative made us own something Red Hat already owns. In an airgapped datacenter, ownership is expensive.

	UPSTREAM / COMMUNITY KNATIVE	OPENSIFT SERVERLESS OPERATOR
Install and upgrades	Helm or YAML we maintain, version by version	An OLM Subscription; the catalog carries the upgrade
Ingress	Pick and run kourier or Istio ourselves	Kourier, managed, in knative-serving-ingress
Routes and TLS	Hand-made Routes with RBAC to write them	Operator-created, edge-terminated, no routes RBAC for us
Airgap	Mirror each image by hand	oc-mirror the catalog, once
Support	Community issues and our own patches	Red Hat, on the OpenShift we already run



Our chart assumes the operator's conventions. Kourier in knative-serving-ingress, operator-managed Routes, DomainMapping for hosts. The API never creates a Route and holds no routes RBAC: that is the operator's job.

02

The API

A Python control plane that turns one HTTP call into a workload on every cluster.

The API: one call, every cluster

Customers describe a workload. The API turns it into Knative objects on both clusters and reports back a single status.

2

offerings: functions and containers

202

every write is asynchronous

0

kubect!. Customers never see Kubernetes

2

regions in every deploy



What it hides

- Knative Services and DomainMappings
- kpack Images, git and registry secrets
- Namespaces, RBAC, NetworkPolicies
- Two clusters that must agree



What it guarantees

- Validation up front, then 202 with a statusUrl
- A closed status vocabulary: Pending · Building · Deploying · Ready · Failed · Terminating
- Machine-readable reasons and one error envelope with a request id



What it is built with

- FastAPI + Pydantic v2, the official kubernetes client
- cloudlet-apis for auth, errors, logging, docs
- 37 test modules, ruff, layering tests
- One Helm chart, rendered by ArgoCD once per region

The API structure

Everything hangs off one base path. The SSO group is a path segment on every workload call, never a body field.

```
/api/serverless/v1
├─ /groups/{group}/functions
│   POST                create → 202
│   GET                 list  ?sort=name|createdAt
│   GET    /{name}      read, secrets redacted
│   PUT    /{name}      full replace → 202
│   DELETE /{name}      → 204
│   POST   /{name}/build rebuild → 202
│   GET    /{name}/stats · /stats/stream
│   GET    /{name}/pods  · /logs/pods/{pod}
├─ /groups/{group}/containers  same set; /pull instead of /build
├─ /functions/info · /containers/info  public capabilities
├─ /stream-tickets  HMAC ticket for EventSource
└─ /healthz · /readyz · /docs · /openapi.json
```



23 endpoints

10 for functions, 10 for containers, two public info endpoints and the ticket mint.



One header, two credentials

A Keycloak OIDC bearer token, validated offline against cached JWKS, or a static admin key, told apart by shape.



403 outside your {group}

Authorization is the groups claim. Names are normalized in one place; admin groups may act for anyone.



Streams get tickets

EventSource cannot send a header, so the browser gets a 60-second, single-path HMAC ticket instead.

Anatomy of a workload

The same body shape for functions and containers, one closed lifecycle, and a response that is the request with secrets redacted.

THE REQUEST

Identity	<code>name · hostname · regions[]</code>
Source (function)	<code>gitRepo · branch · path · gitToken · runtime · version</code>
Source (container)	<code>image · registryUsername · registryToken</code>
Runtime	<code>env[] with secret flag · files[] text or base64 · port · size small medium large</code>
Scaling	<code>minScale · maxScale · metric concurrency rps cpu memory · target · scaleDownDelay</code>

THE LIFECYCLE



Building only exists for functions. Terminating follows a DELETE.

Failed with a machine-readable reason:

BuildFailed

ImagePullFailed

CrashLooping

ConfigError

ProgressDeadlineExceeded

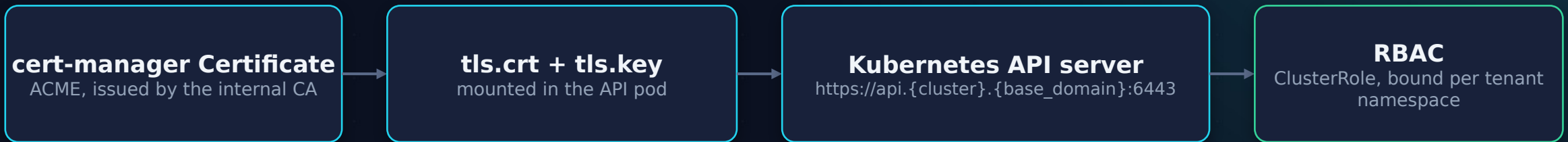


The response

The request with secrets redacted, plus status, hostname, createdAt and one row per region: status, revision, reason, message, replicas. Omit a redacted secret on PUT and it is kept, so read-modify-write never strips credentials.

How the API talks to the cluster

Client-certificate mTLS, always. No kubeconfig, no service-account token, no per-region URL to configure.



CN = `serverless-api.clients.{base_domain}` ← this DNS name is the Kubernetes user



Identity is a DNS name

ACME will only issue to a DNS identity, so the certificate's common name is one. RBAC binds exactly that name; the tenant controller has its own.



Server-side apply, always

Create and update are one code path, with force. A retry after a partial failure simply heals the object; nothing is patched by hand.



A region is three words

{name, cluster, registry}. Nothing secret, so regions ship as a ConfigMap and the API server address is derived, never configured.

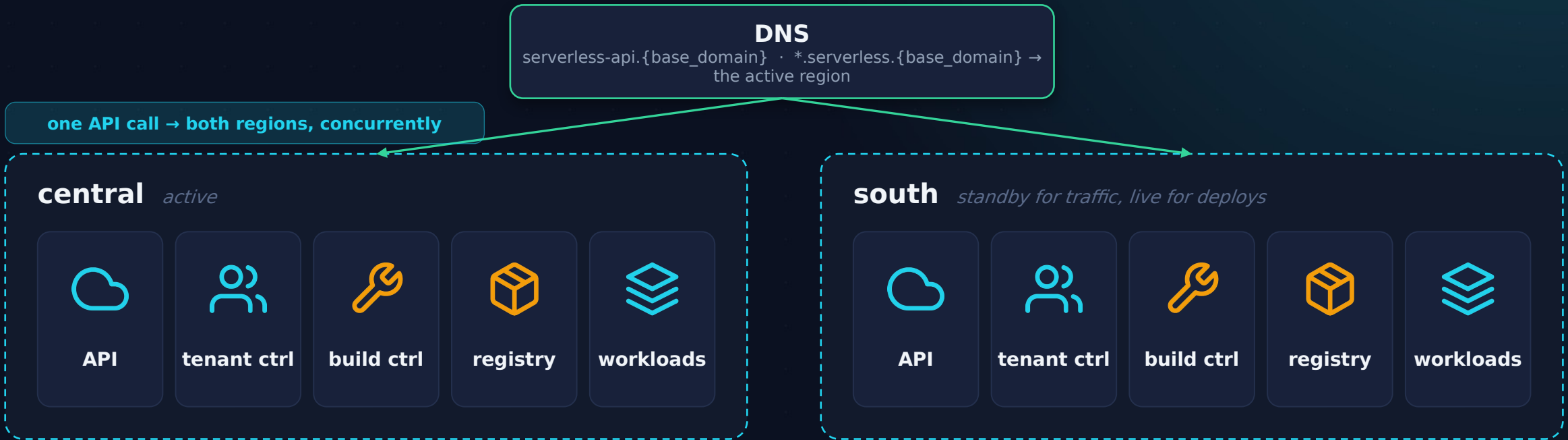


Trust is shared

Both clusters trust the same CA. The OpenShift-injected bundle verifies the servers; connections are lazy, locked and time-boxed.

Active/active: two regions, one truth

The API runs in both clusters and deploys to both. Traffic follows DNS; the state lives in the Knative Service itself.



No leader election, anywhere

Deterministic names, no timestamps or UUIDs in a spec. Two writers applying the same desired state produce one object and one build.

A region builds what it runs

Same commit, different bytes, nothing crosses the boundary at runtime. A switchover needs no reconstruction.

Partial failure stays honest

One region failing yields Failed with that region's reason; the healthy region keeps serving. All failing is a 502, and a read that cannot confirm absence is a 503, never a false 404.

No database

After a switchover, everything is rebuilt from the KSVC annotations, the runtimes ConfigMap and the persisted git Secret.

The tenant controller

A namespace per SSO group, in every region, converged by a separate and more privileged process than the API.



Why a separate process

- Creating namespaces and writing RBAC is cluster-scoped power the internet-facing API must not hold.
- The API may write workloads inside a tenant namespace, but never create a namespace or a NetworkPolicy.
- The controller may do exactly that, and may never touch a workload.
- Own image, own client certificate, own CN. No auth stack inside: CI fails if pyjwt appears.



What lands in {group}-serverless

- The Namespace itself, identical in both clusters
- Default-deny NetworkPolicies, plus one that admits only kpack build pods
- The injected CA-bundle ConfigMap
- A RoleBinding for the API's CN, inside this namespace only, and the SCC binding
- ExternalSecrets for the region registry and the kpack pull credential
- All rendered from template ConfigMaps with {{namespace}} {{group}} {{region}} {{registry}}; a CI job asserts the set is region-neutral.

1 Provision

PUT /groups/{group}/namespace before every accepted deploy. Converges every region concurrently and fails closed: an unreachable controller refuses the deploy with 503.

2 Reconcile

Level-triggered, local cluster only so the two sites never fight, every 300 s, with a forced full converge every twelfth pass to repair drift.

3 Garbage collect

Off by default. A namespace empty of workloads for 24 h is deleted, unless serverless.platform/keep says otherwise.

Stamp protocol: namespace without the hash → contents applied and stale objects pruned → namespace with the template-hash, last. A crash leaves no stamp, so the next pass redoes it.

03

Functions & builds

A container is 'run this image'. A function is 'here is my repo', and that sentence hides a build.

Why functions are different

A container hands us an image. A function hands us a repository, and the image has to come from somewhere.

CONTAINER



FUNCTION



Asynchronous

Minutes, not milliseconds. That is why every write returns 202, why Building is a phase of its own, and why the list shows a build state.



Retried and observable

Each build phase has its own log, a failure is BuildFailed with a message, and POST /build asks for another go without editing anything.



It fires without anyone asking

A CVE patch to the base image or a buildpack rebuilds every function on the platform. Nobody clicks. That is the reason we chose this engine.

Buildpacks and kpack

No Dockerfile. Buildpacks detect the language and assemble the image; kpack runs them as Kubernetes objects we can watch.



Cloud Native Buildpacks

- Detect from requirements.txt, go.mod or package.json. Paketo buildpacks, component by component.
- Layered images: dependencies, runtime and app are separate layers.
- Rebase swaps the run image without a rebuild: a CVE patch in seconds.
- Versions are data:
BP_CPYTHON_VERSION, BP_GO_VERSION, BP_NODE_VERSION from a runtimes ConfigMap.



kpack, in ten CRDs

The ones that matter to us:

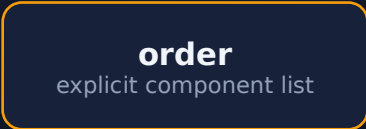


Image declares what to build. Build is one run. ClusterBuilder is the toolchain it uses.



Our kpack repo is a chart

- Not a fork. A Helm chart that packages upstream kpack 0.18 faithfully and adds what an airgap needs.
- CRDs as templates, so the conversion webhook is namespaced correctly.
- clusterBuild creates the ClusterStack, ClusterStore and the credentials that pull them.
- Mirror scripts pull three artifact classes: images, buildpackages, and the runtimes buildpacks fetch at build time.

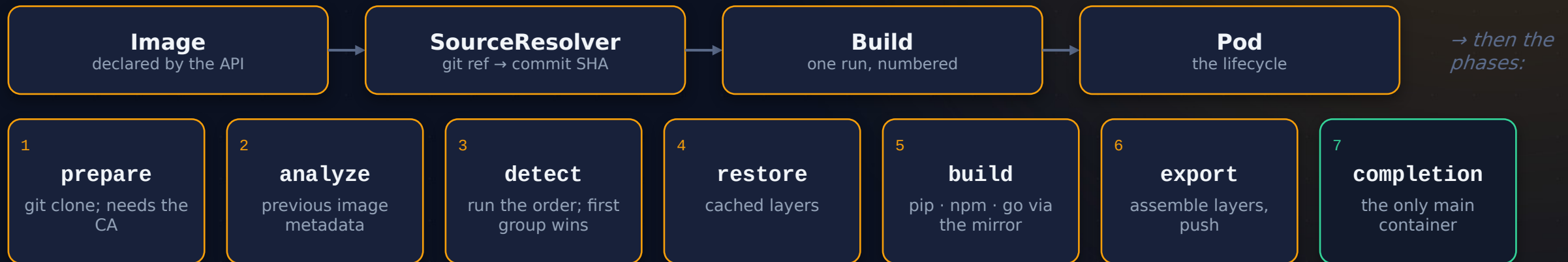


Composing a builder is a push.

The kpack chart owns the read-only, mirror-once content; the serverless chart composes the builders per region, so two clusters never race on one tag.

A build, step by step

Image → SourceResolver → Build → Pod. The lifecycle runs as named init containers, so every phase has its own log.



init containers 1-6 run in order; each is a named container, so `Cluster.pod_logs(pod, container=...)` is a per-phase build log.



What starts a build

- **CONFIG**: a PUT changed the spec. **COMMIT**: the SourceResolver saw a new SHA.
- **TRIGGER**: POST /build annotates the latest Build, never the Image (that would be a nonce).
- **BUILDPACK** and **STACK**: a patched buildpackage or run image. These fire with no user action.

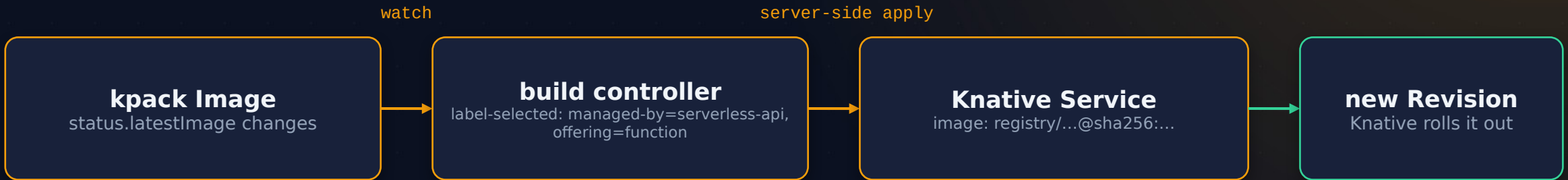


Where it goes, and how it gets out

- `{registry}/{org}/{builderRepo}/{group}/{name}:{branch}`; the layer cache is a registry tag, not a PVC per function.
- A Kyverno ClusterPolicy injects the internal CA, initContainers included: prepare is where the clone happens.
- **BP_DEPENDENCY_MIRROR** → Artifactory with `{originalHost}`: python.org, nodejs.org and go.dev are different hosts.

The build controller

The API declares a build and walks away. The controller watches the result and rolls the digest into the running service.



Both ends are local: the Image is here because this region built it, and the digest names this region's registry. Nothing crosses the boundary.



Exactly one writer per phase

POST writes the branch tag once per region. PUT keeps each region's own value. POST /build writes no Ksvc at all. After that, the controller is the only writer, and only ever a digest.



It knows when to refuse

No write when the Ksvc already runs that digest, or is not labelled offering: function. The apply is a full SSA of the live object stripped of server-owned metadata.



Its own image, no web stack

Installs cloudlet-apis bare. CI imports the service out of the image and fails if fastapi, uvicorn, jwt or cryptography are inside. What is not installed cannot be flagged.



It cleans up after kpack

Sweeps kpack's per-build tags in its own registry every 6 h. Deleting a function removes both repositories in every region through the Quay API.

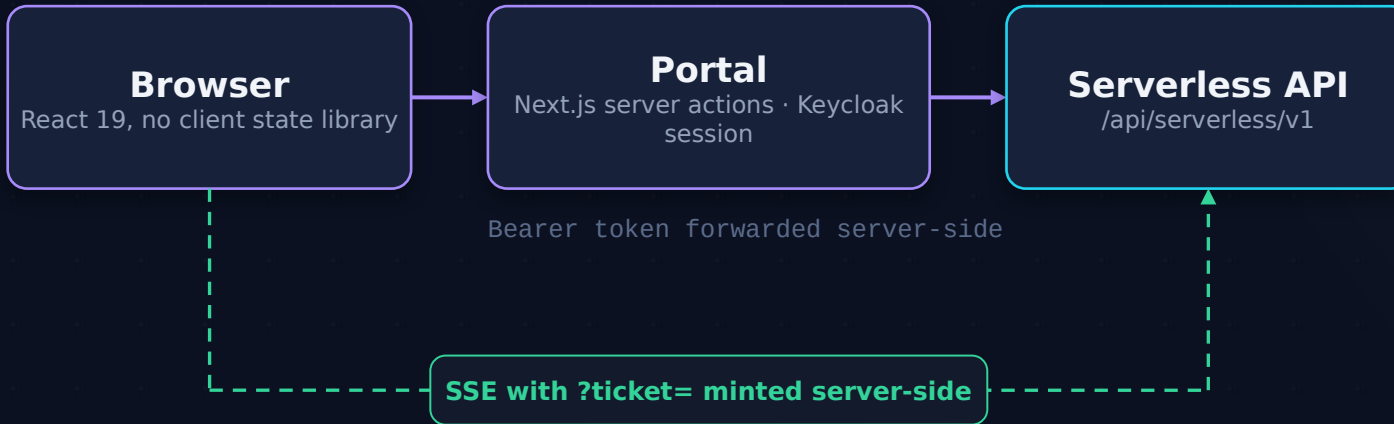
04

Portal & the future

A console for the whole platform, and a library so the next API starts at step five.

How the portal wires it all to the user

A GCP-style console: Next.js 16 server components in front of the API, with the SSO group playing the part of the project.



The user's token never reaches the browser. Live logs and stats use a 60-second single-path ticket that the portal re-mints on every reconnect.



Airgap-native by construction. No CDN, inline SVG icons, hand-written CSS, an internal npm mirror, a Trivy-gated image and a chart with a default-deny NetworkPolicy.



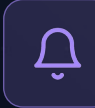
Groups are the projects

Membership is re-validated on every request; a forged cookie can never widen access.



Create dialog built from /info

Sizes, regions, runtimes, port bounds, host preview. The portal hardcodes almost nothing.



202 becomes a tracker toast

A toast polls /stats every 2.5 s until Ready or Failed; a not-found inside the grace window is the race, not an error.

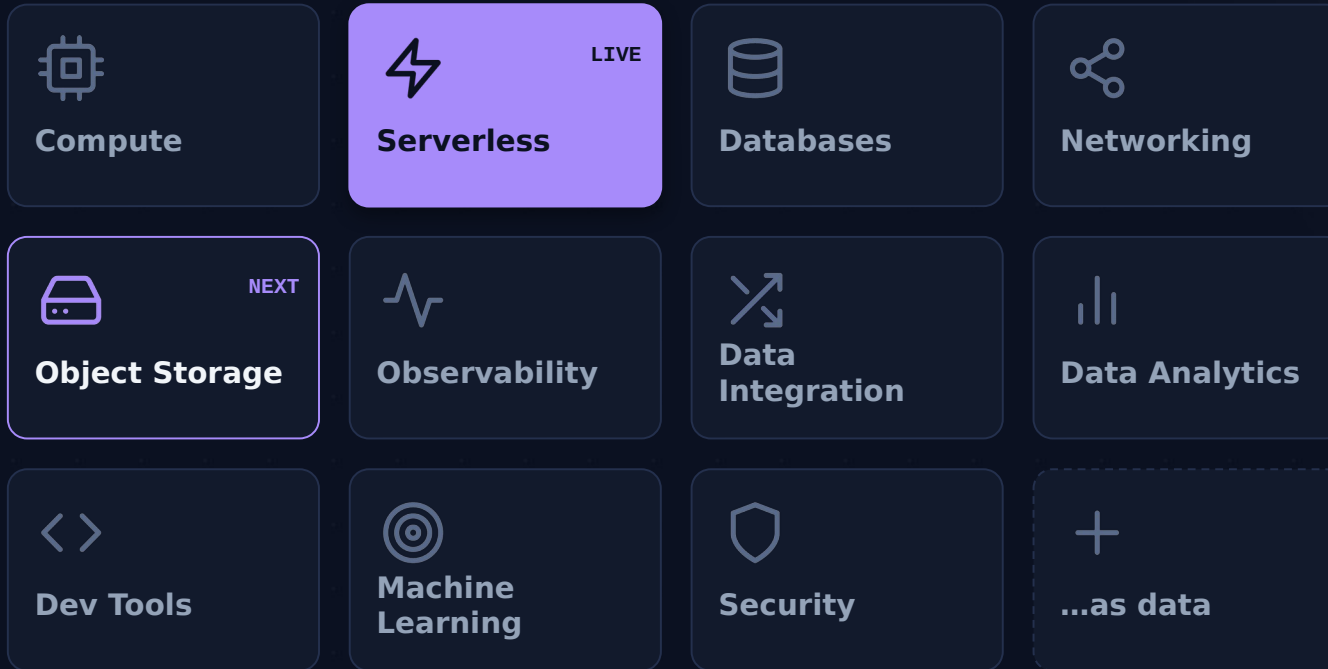


Seven-tab detail, live logs

Status per region, Metrics, Variables, Secrets, Files, Advanced, Logs. Streams fall back to polling and say so.

A console, not a Serverless UI

The service catalog already names the platform we intend to be. Each card lights up the day its API lands.



Object Storage already has its hook: `PORTAL_STORAGE_API_URL`. Set it, and the card is a product.



An offering is data, not code

`PORTAL_SERVICES` JSON or `PORTAL_<NAME>_API_URL` adds a card, a route and nav entries. Unknown ids are appended.



The shell is already generic

`TopBar`, `SideNav`, `auth`, the group cookie, theme, icons. Only the serverless tree and its client are product-specific.



The reusable shape is legible

A `lib/<id>.ts` client, an `actions.ts`, one context gate per product, and forms driven by that API's `/info`.



What we do next

Lift a shared resource layer out of the serverless tree, and generate the TypeScript client from `/openapi.json` so the contract stops being copied by hand.

cloudlet-apis: the next API starts at step five

Everything we got right building the Serverless API, published as a library instead of copied into the next repo.

[auth]

adds FastAPI, httpx, pyjwt

OIDC discovery + JWKS cache · admin key on the same header · SSO login in Swagger · token proxy · stream tickets

[web]

adds FastAPI

health probes · one error envelope · offline Swagger and ReDoc · base_path for a shared host

core

pydantic only

APIError catalogue · Name and Group rules · normalize_group · logging · X-Request-ID

Enforced, not documented: a test spawns a fresh interpreter per core module and asserts FastAPI never appears. The build controller ships with no web stack because of it.

WIRING A NEW API

- 1 Settings** your BaseSettings embeds SSOConfig; the library never reads the environment
- 2 Auth** SSOAuth(settings.sso) and a CurrentUser dependency
- 3 App** logging, RequestIDMiddleware, exception handlers, offline docs, SSO login, health router
- 4 Errors** subclass APIError; the catalogue publishes itself on /info
- 5 Streams** StreamTickets, only if you serve SSE

≈ 15 lines of wiring, plus the security review that came with them: constant-time compares, a grant whitelist, body caps, uniform 401s, discovery backoff.



Why it matters. Two APIs normalizing groups differently is an authorization bug, not a formatting one. Next to extract: the mTLS multi-cluster client and the region fan-out.

WHAT WE BUILT

Code in, a running service in two regions out.

4

repositories

3

services, 3 images

23

API endpoints

2

regions,
active/active

3

runtimes: go ·
python · node

0

kubectl for
customers

- The operator owns Knative; we own the contract on top of it.
- The spec is a pure function of the definition, so two regions need no leader.
- Builds are declared, watched, and rolled in by a controller that can only write digests.
- The portal and the library exist so the second product is cheaper than the first.

**Thank you.
Questions?**